Assignment 2: Break It, Then Build It

A SmartLock Cryptographic System Engagement

Deadline: As per GCR

Instructions

- **This is an individual assignment.** All work must be your own.
- **Language.** This assignment is **C++ only**. All submitted code must be C++17 and must build with `g++` and OpenSSL (`libcrypto`). Work in any other language is not accepted. The one exception is a documented tool sequence for a Part A exploit, as described in A.2.
- **File naming.** Name every file you write as `ROLL-NUM-SECTION-TASK#-VERSION.EXT`, for example `23i-0001-A-TASKA3-V1.cpp` for your keying exploit. The task codes are `TASKA1` ... `TASKA5` for the five Part A exploits (in the order downgrade, impersonation or man in the middle, keying recovery, nonce reuse, replay) and `TASKD` for your performance tooling. **One exception:** your completed Part C module keeps the exact name `securelink2.cpp`, and any v2 scaffold file you change keeps its original name, because the submission check compiles them by name. Each code file, including `securelink2.cpp`, must have your **name, student ID, and assignment number** in a comment at the top.
- **Folder and archive.** Submit one folder, laid out exactly as shown under "Deliverables", with no binaries, `.o`, `.exe`, or other build output. Name the folder `ROLL-NUM-SECTION` (for example `23i-0001-A`) and submit it as a `.zip` file (for example `23i-0001-A.zip`). Do **not** submit a `.rar` file.
- **Where to submit.** All submissions must be made on **Google Classroom** within the deadline. Submissions by any other means (for example email) will not be accepted.
- **Your archive is your responsibility.** You are solely responsible for checking your final zip for problems such as corrupt files, viruses, or executables. If the file cannot be downloaded from Google Classroom for any reason, the assignment receives zero marks.
- **It must compile.** If a submitted source file does not compile with `g++` (a syntax error, a missing include you relied on), that part of the assignment receives zero marks. State your exact build command in your report.
- **Presentation.** Program output should be well formatted and clearly presented. Use appropriate comments and indentation in your source code.
- **Evaluation and viva.** Understanding the assignment is part of the evaluation. Be prepared for a viva or any other evaluation mechanism after submission; you must be able to run your own exploits and explain your own code on request.
- **Policies.** Late submission and plagiarism policies apply as described in the course outline.
- **Tip.** For timely completion, start as early as possible. The lab setup and the first capture take longer than you expect.

---

1. The scenario

The product. A vendor ships a smart door lock, **SmartLock**. It is a small Internet-of-Things device bolted to a front door: a motorised dead bolt with a tiny processor and a network radio, and a label on the back printed with a **pairing PIN**. It is sold with a **home hub**, an always-on box that sits

somewhere central in the house (think of a Wi-Fi router or a smart-home controller) and speaks to the lock on the owner's behalf. The marketing claims the product protects every stage of a session: it negotiates a cipher suite, authenticates the lock and pairs the hub, performs a Diffie-Hellman key exchange, encrypts every command, and closes the session cleanly. A home owner who installs it is trusting it with the physical security of their front door, so the promise that nobody on the network can read the traffic, impersonate the lock, or replay an old "unlock" is exactly the promise you are asked to test.

How the two devices talk. The lock and the hub are two separate programs that communicate over the **local network** using ordinary **TCP/IP sockets**, the same transport that carries web and email traffic. The relationship is **client-server**: the **lock is the server** (it runs `lock_server`, binds to a TCP port, and waits), and the **hub is the client** (it runs `hub_client`, opens a TCP connection to the lock's address and port, and drives the conversation). One session is one TCP connection, opened when the hub wants to act and closed when it is done. On top of TCP the two sides speak a simple **line-based, request-and-response protocol**: each step is a single line of text, the hub sends a line and the lock answers, and the exchange always runs through the same five stages in order:

1. **Negotiation**: the hub proposes which cipher suite to use and the lock chooses one.
2. **Authentication**: the lock presents a certificate that is meant to prove which lock it is, and the hub proves it is an authorised controller by showing it knows the pairing PIN.
3. **Keying**: the two sides run a Diffie-Hellman exchange to agree on a shared secret key for the session.
4. **Message protection**: every command and reply after that is an encrypted record.
5. **Teardown**: the hub sends `QUIT` and the session is closed.

Because this all happens over the home network, anyone who can see that network, a guest on the Wi-Fi, a neighbour who has broken in to it, or a device already compromised elsewhere in the house, is a realistic attacker in this scenario. That is the position you will model in the lab.

What it can do. Once the hub is paired, the owner can ask the lock to do a small, fixed set of things. The hub sends `STATUS` to read the current state, `UNLOCK` to open the door, `LOCK` to secure it, and `QUIT` to end the session. The lock keeps two pieces of state so the effect of each command is visible: a **door state** (`LOCKED` or `UNLOCKED`) and an **unlock counter** that rises by one every time the door opens. Both last for as long as the lock program runs, so they carry over from one session to the next and reset only when you restart `lock_server`. As soon as the session key is agreed, the lock also sends a one-line **greeting** that carries its **pairing secret**; in this assignment that secret is your **flag**, a value of the form `FLAG{...}` unique to your instance, and recovering it from the network is one of your goals. The precise wire format of every message is in `FUNCTIONALITY.pdf`; read it before you start.

You are hired to review this system. You will work in two roles, one after the other.

First, as an **attacker**, you take the shipped version (v1) apart stage by stage. You capture its traffic on an isolated lab network, you reason about what each message reveals, and you prove each weakness with a working exploit, not with a guess or a pointer at a line of source. At least one of your exploits must recover your **flag** (the device's pairing secret).

Then, as a **designer**, you specify and build a corrected version (v2). You state the threat you are defending against, choose concrete mechanisms for every stage, implement them, and then prove by testing that each of your five exploits now fails. Finally you measure what your security costs, because a lock that is secure but too slow to use is its own kind of failure.

Keep two references beside you as you work: `FUNCTIONALITY.pdf`, which gives the exact wire format of every message, and the `Appendix`, which explains the families of attack you are likely to use, since you have met these ideas in lectures but not yet with live tools.

2. Learning outcomes

1. Analyse a cryptographic system stage by stage and map each weakness to the security property it breaks (confidentiality, integrity, authentication, non-repudiation, forward secrecy).
2. Capture, analyse, and exploit protocol traffic with standard tools.
3. Design a system that is sound at every stage and justify each choice against a stated threat model.
4. Implement and test a secure protocol, and measure the cost of its security.

3. What you are given, and the environment

In `smartlock/`: the full v1 source (`slink.hpp`, `pki.hpp`, `lock_server.cpp`, `hub_client.cpp`), the `instance_generator` you run to create your own instance, and a `Makefile`. In `v2_skeleton/`: the skeleton you complete for Part C, with its own `README.md`. At the top level: this statement, the `Appendix`, `FUNCTIONALITY.pdf`, the `Lab_Setup_Guide`, `answers_template.txt`, and `BUILD.md` (the toolchain and build commands). You create your own `instance.txt` from your roll number (section 4).

The v1 source is provided so you understand the system, but Part A marks are for weaknesses you **find and exploit on the wire**, not for weaknesses you only point at in the code.

Use a recent `g++` (version 11 or newer, C++17) and OpenSSL's development headers. On Kali or Debian/Ubuntu:

```
sudo apt install build-essential libssl-dev
```

Run the attacks from a Kali Linux VM. Because a network attack needs two separate hosts (traffic between a client and server on the same machine never reaches the wire and cannot be intercepted), run the lock and the hub on **different virtual machines on an isolated host-only network**. The Lab Setup Guide gives two layouts: a two-VM minimum (lock on its own VM, Kali runs the hub and the tools, with on-path attacks modelled by a local relay) and a three-VM full setup (lock, hub, and a Kali attacker doing a real ARP-spoofing man in the middle). Keep all interception and spoofing inside this lab, and include a short "Scope and ethics" paragraph in your report confirming that you did.

4. Your instance and how it is generated

You create your own `instance.txt`, a simple `key = value` file that is the lock's complete configuration: its device name and RSA key, the cipher suites it offers, its Diffie-Hellman group and public value, the pairing PIN (and its digest), the record nonce, the TCP port, and your pairing secret (your **flag**). Build the tools and generate it once, using **your own roll number**:

```
cd smartlock
make
./instance_generator 23i-0001      # use your roll; writes instance.txt
```

The generator is **seeded from your roll number**. It computes a SHA-256 digest of your roll number and uses that digest to seed a deterministic pseudo-random generator; every parameter in your instance is then derived from that single seed. Two consequences follow, and both matter for grading:

- **Your instance is unique.** A different roll number seeds a different generator and therefore a different Diffie-Hellman group, a different device key, a different PIN, a different nonce, a different flag, and a different port. No two students attack the same lock.
- **Your instance is reproducible.** Because the derivation is deterministic, the instructor can regenerate your exact instance from your roll number alone and check that every value you report is genuinely yours. **A result that matches another student's instance, or any generic value, scores zero on the affected part.** Every recovered number you submit must be the one that belongs to your own `instance.txt`.

You run your own lock with your own `instance.txt`, exactly as the owner's device would run in the field. Treating the protocol as a black box on the wire, your task is to show how much of that configuration a network attacker can recover without ever opening the file.

5. Tools

Suggested, not exhaustive: **Wireshark / tshark / tcpdump** for capture; **Scapy / mitmproxy / bettercap / ettercap** for interception and modification; **a short socket client** for replay (see the Lab Setup Guide on why **tcpreplay** alone is not enough); **hashcat / John the Ripper** for the pairing PIN (a wordlist, `smartlock/pin_wordlist.txt`, is provided); and **openssl / CyberChef** together with your own short C++ programs for the cryptographic steps. The Appendix explains what each of these is for. A cryptographic weakness is found by capturing traffic, reasoning about it, and writing a short program, not by running one scanner: show the command, the capture, and the code.

Part A: Break SmartLock v1 (32 marks)

Analyse the system stage by stage: **negotiation**, **authentication**, **keying**, **message-by-message protection**, and **teardown**. The Appendix describes each attack family and the tools in detail.

A.1 Recovered secrets (10 marks). Submit an `answers.txt` (a `key = value` file; the format is shown in section "Deliverables") containing the values you recovered **from your own instance, over the wire**:

- your **flag**, the pairing secret, recovered by any path (4);
- the device's Diffie-Hellman **exponent** `b`, found by discrete log, such that `g^b mod p` equals the captured `B` (3);
- the cracked **pairing PIN** (2);
- optionally, the **nonce-reuse XOR** (1): the hex of the greeting plaintext XOR a `STATUS` reply plaintext from the same session. Give at least the first 8 bytes, and no more than the length of the `STATUS` reply.

These four values are also checked at the **viva**: you will be given a capture from an **unseen instance** (not your own) and asked to recover the same values with the tools you submitted. Running the autograder on your `answers.txt` confirms the values are genuinely yours; the viva confirms that you can actually carry out the recovery. Write your exploits to read values from a capture, not to hard-code your own numbers. See Part E.

A.2 Five-stage exploit demonstrations (22 marks). For **each** of the five stages, present four things: the **weakness** in one or two sentences; the **evidence** (a packet capture or tool output that shows it); a **working exploit** (a C++ program or a documented tool sequence, see "Deliverables") and what it achieves; and the **impact** (the security property that fails). At least one exploit must recover your flag. The five exploits are worth 4 marks each (downgrade; impersonation or man in the middle; keying recovery; nonce reuse; replay, which is the exploit for the teardown stage); the remaining 2 marks are for recovering the flag and for the clarity of your Part A write-up. Mapping each exploit to the property it breaks is explicitly part of the marks, so tie every finding to confidentiality, integrity, authentication, non-repudiation, or forward secrecy.

Part B: Design SmartLock v2 (18 marks)

Write a short, concrete design specification before you write code.

- A **threat model** (4): state precisely what your attacker can and cannot do (what they see, what they can change, what secrets they lack) and what security goals v2 must meet.
- A **per-stage mechanism** for all five stages (10, two marks each): for every stage name the concrete mechanism you will use and tie it to the specific Part A attack it defeats. Name real choices: an authenticated key exchange, a key derivation function, an AEAD mode with an explicit nonce rule, a sequence-number scheme, and a defined, authenticated close. Your authentication mechanism must also say how v2 should protect the pairing PIN, both at rest and against an attacker who captures the pairing. The provided scaffold does not fix this (see C.2), so this is a design requirement only.
- A **v2 handshake diagram** (4): one clear diagram of the corrected exchange, message by message, showing where authentication, key derivation, and sequence binding happen.

Part C: Implement and test SmartLock v2 (28 marks)

Complete `v2_skeleton/securelink2.cpp`, keeping every function signature declared in `securelink2.hpp` (its `README.md` gives the contracts). This module is the security core: the suite policy, the key derivation from the Diffie-Hellman secret, the per-record nonce, and the sequence-bound AEAD. The provided `lock_server2.cpp` and `hub_client2.cpp` already supply the strong ephemeral Diffie-Hellman exchange, the signed and pinned certificate, and the record loop; they call into your module, so once it is complete they build and run.

- **C.1 Correctness of the five fixes (18 marks).** Your `securelink2.cpp` must pass six checks worth 3 marks each, run automatically on submission (see the self-check list under "Deliverables"): it never selects `NULL` and never downgrades; it derives the record key from the shared secret with a KDF rather than using the raw secret; it uses a unique nonce per record; its AEAD seals and opens a record correctly; and it rejects a replayed or out-of-order record. The sixth check is a source scan: the file must contain one of `hkdf`, `evp_kdf`, `pbkdf2`, or `scrypt`, and must not contain `md5` (in any letter case) or `TODO`, including in comments.
- **C.2 Your own testing (10 marks).** Re-run each of the five A.2 exploits against your v2 and show it now fails: a table mapping "v1 attack to v2 result", with the commands and the output. **PIN cracking is not part of this table:** the provided scaffold still sends the v1 MD5 `pinhash` in `PAIR`, so a captured v2 pairing can still be cracked offline. That is a known limitation of the scaffold, which you address in your Part B design and do not have to implement. Show that a normal session still succeeds and that a tampered or replayed record is rejected. Include the **AI-audit component**: ask an AI tool to write one piece (for example the record layer), then

state in your own words what was weak or wrong in the cryptography it produced and how you corrected it.

C.2 marks: the five re-run exploits, 1 each (5); a normal session plus tampered and replayed records rejected (2); the AI-audit component (3).

Part D: Performance study (8 marks)

Measure what your security costs and report it honestly. Your **baseline** is v1 under `AES256-GCM`, measured on the same machine with the same compiler flags as your v2. Report these three measurements for both versions:

- **Handshake time:** from opening the TCP connection to having the greeting decrypted. Use at least 30 runs.
- **Per-record overhead:** the extra bytes on the wire for a record compared with its plaintext, and the time to protect and to open one record.
- **Throughput:** MB/s protecting and opening 1 KiB records, using at least 10,000 records per run and at least 5 runs.

Give the mean and standard deviation of each timing, and state your machine and your method. Present a table and one chart (5); the chart may be made with any tool. Add a paragraph that defends the trade-off (3): what the extra cost buys, and why it is the right choice for a device that controls a door. You are graded on honest relative numbers and clear reasoning, not on absolute speed.

Part E: Viva (10 marks, mandatory)

Each student attends a short oral defence. You will explain parts of your own exploits and code (4), make a small live change, for example show what leaks if a nonce repeats in your v2 (3), and justify a design choice (3). You must be able to run your own exploits on request, and to recover the Part A.1 values from a capture of an **unseen instance** using the tools you submitted. **Without the viva, Parts A to D are not credited.**

Deliverables

Submit `ROLL-NUM-SECTION.zip` (for example `23i-0001-A.zip`) containing a single folder with this layout. Leave out any optional folder you do not need.

```
23i-0001-A/
  23i-0001-A-REPORT.pdf
  answers.txt
  README.txt
  securelink2.cpp
  exploits/
    23i-0001-A-TASKA1-V1.cpp      downgrade
    23i-0001-A-TASKA2-V1.cpp      impersonation or man in the middle
    23i-0001-A-TASKA3-V1.cpp      keying recovery
    23i-0001-A-TASKA4-V1.cpp      nonce reuse
    23i-0001-A-TASKA5-V1.cpp      replay
  v2_changes/                     optional
  performance/
    23i-0001-A-TASKD-V1.cpp
    (raw benchmark data and your chart)
```

```
  evidence/
    (captures, tool output, and git_log.txt if you used version control)
```

1. **Report** (`ROLL-NUM-SECTION-REPORT.pdf`), covering Parts A to D, with evidence inline (captures, tool output, and your numbers), the "v1 attack to v2 result" table, the "Scope and ethics" paragraph, and your flag.
2. `answers.txt` for Part A, as a `key = value` file in the format shown below.
3. `README.txt` : a short explanation of how to build and run your exploits and your v2.
4. `securelink2.cpp` : your completed Part C module, at the top level of the folder and under exactly this name.
5. `exploits/` : one file per Part A exploit. Use a C++ program, or, for an exploit you carry out with standard tools, a `.txt` file with the same task name that lists the exact commands, including any filter or addon the tool needs. You may add shared headers here, named `ROLL-NUM-SECTION-TASKA-*.hpp` .
6. `v2_changes/` (only if you changed them): your versions of `lock_server2.cpp` and `hub_client2.cpp` , under their original names.
7. `performance/` : your Part D tooling, raw data, and chart.
8. `evidence/` : your captures and tool output, including `.pcap` files.

The format of `answers.txt` (also in `answers_template.txt`):

```
roll           = 23i-0001
flag           = FLAG{...your recovered pairing secret...}
recovered_dh_b = 0
cracked_pin    = ...
nonce_xor_hex  =            # optional
```

Keep a real, incremental commit history if you use version control. Do not submit the `.git` folder; save the output of `git log --stat` as `evidence/git_log.txt` instead. A single final dump is treated as a flag for review.

Report structure

Use these sections, in this order, and keep the report to **at most 20 pages**: title page (name, roll, section, build commands); scope and ethics; Part A, one subsection per stage; Part B; Part C, including the "v1 attack to v2 result" table and the AI-audit component; Part D. The 4 report marks are for following this structure (2), evidence placed inline where it is discussed (1), and concision (1).

Self-check before you submit

There is no grader to run; the checks below are exactly what is verified on submission, so confirm each one yourself with your own test output and include that output in your report.

- **Part A:** every value in `answers.txt` was recovered from your own instance on the wire, and `g^(recovered_dh_b) mod p` equals your captured `B` .
- **Part C:** build `v2_skeleton` cleanly with `make` ; a normal `lock_server2` / `hub_client2` session succeeds; your `securelink2.cpp` refuses a `NULL` -only offer, derives the key with a KDF (not the raw secret), uses a fresh nonce per record, seals and opens a record correctly, and throws when a record arrives at the wrong sequence number, and passes the source scan described in C.1.

Marking scheme (every line is checkable)

Tags: **[AUTO]** a mechanical check the instructor runs on your submission; **[VIVA]** confirmed in the oral defence; **[EVID]** judged against your submitted evidence and cross-checked with the automatic checks and the viva.

Part	Criterion	Check	Marks
A.1	Flag matches your instance (and recovered live at the viva)	[AUTO]+ [VIVA]	4
A.1	Recovered DH exponent satisfies g^b = B (and recovered live at the viva)	[AUTO]+ [VIVA]	3
A.1	Cracked pairing PIN matches (and recovered live at the viva)	[AUTO]+ [VIVA]	2
A.1	Nonce-reuse XOR matches (optional)	[AUTO]	1
A.2	Negotiation downgrade exploit (capture + working PoC)	[EVID]+ [VIVA]	4
A.2	Authentication impersonation or MITM (relay demonstrated)	[EVID]+ [VIVA]	4
A.2	Keying recovery (discrete log, key derived, flag read)	[EVID]+ [VIVA]	4
A.2	Nonce-reuse exploit (P1 xor P2 recovered)	[EVID]+ [VIVA]	4
A.2	Replay exploit (captured UNLOCK re-opens the lock)	[EVID]+ [VIVA]	4
A.2	Part A report hygiene and flag write-up	[EVID]	2
B	Threat model stated	[EVID]+ [VIVA]	4
B	Per-stage mechanism and justification (2 each)	[EVID]+ [VIVA]	10
B	v2 handshake diagram	[EVID]	4
C.1	Six v2 checks verified (3 each)	[AUTO]	18
C.2	v1 attacks re-run and shown to fail; tamper and replay rejected; AI-audit component	[EVID]+ [VIVA]	10
D	Performance table and chart over several runs	[EVID]	5
D	Trade-off paragraph	[EVID]	3
E	Explains own exploits and code	[VIVA]	4
E	Correct live modification	[VIVA]	3
E	Justifies a design choice	[VIVA]	3
Report	Structure, evidence, concision	[EVID]	4
	Total		**100**

[AUTO] marks are all-or-nothing per line. [EVID] and [VIVA] marks require the work to be demonstrable on request; work you cannot run or explain is not credited.

Academic integrity and AI use

You may use AI to learn and to draft code, but you are responsible for everything you submit and must defend all of it in the viva. Two rules keep this fair: your results must match **your own instance** (a generic answer does not fit your numbers), and in the viva you must explain and modify your own work on request. Part C.2 deliberately turns AI use into part of the exercise: you audit what an AI produces rather than trusting it.